

SIVF: GPU-Resident IVF Index for Streaming Vector Analytics

Anonymous Author(s)

Abstract

GPU-accelerated Inverted File (IVF) index is one of industry standards for large-scale vector analytics but relies on static VRAM layouts that hinder real-time mutability. Our benchmark and analysis reveal that existing designs of GPU IVF necessitate expensive CPU-GPU data transfers for index updates, causing system latency (at the granularity of 10K 128-dimensional vectors) to spike from milliseconds to seconds in streaming scenarios. We present SIVF, a GPU-native index that enables high-velocity, in-place mutation via a series of new data structures and algorithms, such as conflict-free slab allocation and coalesced search on non-contiguous memory. SIVF has been implemented and integrated into the open-source vector search library, Faiss. Evaluation against 5 baselines (GPU IVF, GPU Flat, HNSW, NSG, LSH) with 5 vector datasets (synthetic, Deep1B, SIFT1M, T2I-1B, GIST1M) suggests that SIVF effectively removes the I/O bottleneck of GPU indices for streaming vectors, therefore transforming vector search from a static retrieval task into an integral component of real-time vector data analytics.

1 Introduction

Real-world vector analytics applications, ranging from search engines to dynamic retrieval-augmented generation (RAG) of large language models (LLMs), increasingly operate on streaming data where timeliness is critical [29, 37]. These systems necessitate a sliding-window model where expired vectors must be invalidated promptly as new vectors arrive to maintain bounded memory footprint. However, existing GPU-accelerated approximate nearest neighbor (ANN) indices are predominantly optimized for static, write-once-read-many workloads [32, 54].

To quantify the gap between insertion and eviction performance, we conducted benchmarks with the industry-standard Faiss library [20] on an NVIDIA RTX 6000 GPU hosted at the Chameleon Cloud [23]. We utilized the SIFT1M dataset [22] (128-dimensional) to simulate a realistic sliding-window scenario. Specifically, we initialized a standard IVF1024 index pre-populated with 100,000 vectors and measured the wall-clock latency of ingesting a batch of 10,000 new vectors and subsequently evicting the oldest 10,000 vectors. As illustrated in Figure 1, we observe a drastic performance asymmetry between insertion and deletion performance. While GPU parallelism accelerates the insertion of the 10K-vector batch to approximately 28 ms, the eviction process for the same batch size is significantly slower, spiking the latency to ~200 ms. This high latency is observed in the native C++ implementation, which performs nearly identically to the Python binding. The negligible difference between the compiled C++ core and the interpreted Python interface suggests that the bottleneck is not an implementation detail: If the issue were merely inefficient code, the C++ implementation would be expected to significantly outperform Python. The lack of such divergence suggests that both are bound by the same architectural limitation: the absence of a GPU-resident deletion operator, which forces expensive CPU-GPU roundtrips for eviction.

High Deletion Overhead: Python vs. C++

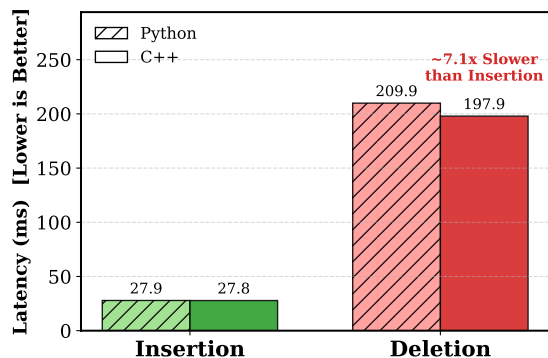


Figure 1: The asymmetry between insertion and deletion.

Our analysis of the Faiss source code [6] further reveals that this performance asymmetry stems from a rigid class hierarchy that enforces a fallback to host-side processing on CPUs. In the library’s architecture, the data eviction interface is defined in the abstract base class `faiss::Index` via the `remove_ids` virtual method. While CPU-based implementations override this method to provide efficient in-memory deletion logic (e.g., `memmove`), the GPU counterparts inherit the base implementation which lacks native support. Consequently, current GPU indices rely on a *CPU-GPU Roundtrip* pattern for eviction. That is, to delete even a single vector, the system must transfer the entire index state from VRAM to host memory, perform the compaction on the CPU, and re-upload the modified index to the device. This heavy I/O burden saturates the PCIe bus and prevents the system from utilizing the GPU’s compute throughput, capping the maximum sustainable ingestion rate in streaming scenarios.

The absence of a GPU-resident eviction operator is not an oversight but a consequence of the complexity involved in managing dynamic data structures within VRAM. Unlike CPU indices that utilize dynamic arrays, GPU indices rely on pre-allocated, contiguous memory blocks to maximize memory coalescing and bandwidth utilization. Supporting in-place deletion on GPUs presents at least 3 challenges. First, maintaining contiguous arrays requires dynamic memory management. Frequent reallocation and compaction of inverted lists within VRAM lead to severe memory fragmentation and costly data movement. Second, concurrent modification requires complex synchronization. Managing fine-grained locks to prevent race conditions when thousands of concurrent GPU threads attempt to modify shared list structures introduces significant latency. Third, standard inverted file (IVF) indices lack an ID-to-Location mapping. Without a reverse index, locating a specific ID for deletion requires scanning all inverted lists, an operation with $O(N)$ complexity that negates the benefits of GPU acceleration. These constraints make naive GPU eviction prohibitively inefficient.

To resolve these limitations, we propose SIVF, a GPU-resident indexing method that supports high-throughput mutability through four specialized device-side algorithms. First, to enable nonblocking writes, we design a *Lock-Free Ingestion* protocol (Algorithms 1 & 2) that manages a pool of fixed-size slabs. This protocol utilizes atomic Compare-And-Swap for slot reservation and speculative head updates, allowing concurrent threads to append to linked lists without global locks. Second, to guarantee data consistency under CUDA’s relaxed memory model, we apply a strict publication ordering; the kernel issues device-wide memory fences (`__threadfence()`) before committing validity bits, ensuring readers never observe partially initialized payloads. Third, to mitigate the latency of pointer chasing, we propose *Warp-Cooperative Search* (Algorithm 3). By aligning the slab capacity ($C = 32$) with the hardware warp width, this algorithm enables threads to cooperatively load and evaluate non-contiguous memory blocks while preserving coalesced access patterns. Finally, we address deletion by maintaining a GPU-resident Address Translation Table along with lazy eviction (Algorithm 4), allowing SIVF to decouple logical removal from physical data movement, which makes eviction to a constant-time operation.

SIVF has been implemented and integrated into Faiss [6] as a new index through the `GpuIndexSIVF` class. Our implementation leverages CUDA for device-side kernels and employs a dual-view memory management strategy to maintain the consistency between host and device states. All new algorithms as well as key data structures are encapsulated within a `SlabManager` class that abstracts the complexity of dynamic memory operations on the GPU.

We have evaluated SIVF against a comprehensive set of baselines within Faiss, including GPU Flat, GPU IVF, HNSW, NSG, and LSH indices. Our evaluation was carried out on five datasets representing a variety of modalities, dimensions, and scalability: synthetic, Deep1B, SIFT1M, T2I-1B, and GIST1M. Experimental results show that SIVF achieves up to 13,307× reduction in deletion latency (Fig. 8) and improves ingestion throughput by 36×–120× compared to existing GPU indices (Fig. 7). In end-to-end sliding window scenarios, SIVF delivers a 163×–262× speedup (Fig. 10) with a negligible memory overhead less than 0.8% (Fig. 11), effectively closing the gap between static and streaming vector analytics performance.

In summary, this paper makes the following contributions:

- We identify the CPU-GPU bottleneck in existing GPU vector indices when being used for sliding-window scenarios, demonstrating that the lack of native deletion support renders them unsuitable for streaming vector analytics.
- We propose a new indexing method, namely SIVF, that synthesizes a series of GPU-optimized data structures and algorithms for streaming vector analytics. This design enables $O(1)$ time complexity for deletion and constant-time insertion overhead with negligible memory overhead.
- We implement SIVF in the state-of-the-art vector search library Faiss and evaluate SIVF against a broad spectrum of baselines (GPU Flat, GPU IVF, HNSW, NSG, and LSH) on 5 datasets: synthetic, Deep1B, SIFT1M, T2I-1B, and GIST1M. The results demonstrate that SIVF achieves up to 13,307× reduction in deletion latency, 120× faster ingestion, and 266× end-to-end sliding window speedup with less than 0.8% memory overhead.

2 Related Work

Optimization of Graph-Based Indices. Graph-based indices remain the state-of-the-art for high-recall retrieval. To address performance bottlenecks in production, frameworks like VSAG [55] optimize memory access patterns and parameter tuning, while SOAR [43] improves indexing structures. Of note, CAGRA [38] establishes a new state-of-the-art for static graph construction and search throughput on GPUs; however, it is fundamentally designed for write-once-read-many workloads and lacks native support for the efficient, fine-grained in-place deletions required by streaming applications. Navigational efficiency is a key optimization target; SHG [11] introduces a hierarchical graph with shortcuts to bypass redundant levels, and probabilistic routing methods [33] have been proposed to enhance traversal. Several works focus on distance metric optimizations: ADA-NNS [21] utilizes angular distance guidance to filter irrelevant neighbors, DADE [5] accelerates comparisons via data-aware distance estimation in lower dimensions, and HSCG [41] adapts the Monotonic Relative Neighbor Graph for cosine similarity using hemi-sphere centroids. Efforts to improve graph construction and maintenance include LIGS [3], which employs locality-sensitive hashing (LSH) to simulate proximity graphs for faster updates, and CSPG [51], which crosses sparse proximity graphs. Addressing robustness, Hua et al. [13] propose dynamically detecting and fixing graph hardness for out-of-distribution queries. MIRAGE-ANNS [45] attempts to bridge the gap between incremental and refinement-based construction. Furthermore, theoretical frameworks like Subspace Collision [48] provide guarantees on result quality using clustering-based indexing.

Collectively, while these approaches maximize navigational efficiency for static data, they fundamentally rely on complex edge dependencies that are prohibitively expensive to maintain on GPUs. Unlike SIVF, they lack the architectural support for $O(1)$ lock-free mutation, rendering them unsuitable for high-churn streaming workloads where sub-millisecond update latency is critical.

Attribute-Filtered and Hybrid Search. The integration of structured constraints with vector analytics has led to a surge in Filtered ANN research. Li et al. [26] provide a comprehensive taxonomy and benchmark of these methods. A primary focus is handling dynamic updates and specific filter types such as ranges or windows. For range filtering, UNIFY [30] introduces a segmented inclusive graph to support pre-, post-, and hybrid filtering strategies. Dynamic environments are addressed by DIGRA [19], which uses a multi-way tree structure for dynamic graph indexing, and RangePQ [53], which offers a linear-space indexing scheme for range-filtered updates. Peng et al. [39] propose dynamic segment graphs to handle mixed streams of data and queries. Regarding specific constraints, Wang et al. [46] present a robust framework for general attribute constraints. Engels et al. [7] focus on window filters, while WoW [47] develops a window-graph based index to handle window-to-window incremental construction.

However, these contributions primarily optimize algorithmic filtering logic rather than the underlying storage architecture. They remain constrained by the static memory layouts of standard GPU indices, whereas SIVF fundamentally redesigns the VRAM management to enable the high-throughput, in-place mutability that is a prerequisite for such dynamic operations.

Quantization and Scalability. To manage high-dimensional data at scale, quantization and system-level optimizations are critical. RaBitQ [10] and its extended version [9] provide theoretical error bounds for bitwise quantization with flexible compression rates. SymphonyQG [12] integrates quantization codes directly with graph structures to reduce memory access overhead, while LoRANN [18] utilizes low-rank matrix factorization for compression. In terms of system architecture, Tagore [28] leverages GPUs for accelerating graph index construction. For distributed environments, HARMONY [49] employs a multi-granularity partition strategy to balance load. WebANNS [31] enables efficient search within web browsers via WebAssembly. From a theoretical perspective, Indyk and Xu [17] analyze the worst-case performance limits of popular implementations. Finally, DARTH [1] introduces declarative recall targets through adaptive early termination to balance performance and result quality.

While these techniques reduce storage footprints or distribute load, they predominantly operate on static memory layouts that are brittle under frequent updates. SIVF addresses a complementary challenge: it provides the dynamic memory substrate that allows these scalable architectures to support real-time streams without succumbing to VRAM fragmentation or locking overheads.

GPU Acceleration Techniques. Recent HPC work highlights how careful kernel and data-layout design can unlock GPU performance across data-intensive workloads. FZ-GPU demonstrates a fully parallel compression pipeline with both warp-aware bitwise operations and shared-memory efficiency [52]. For sparse linear algebra, SpMV designs reduce preprocessing overhead while improving load balance and memory access locality [2], and GPU-aware preconditioning further emphasizes locality and coalescence as first-order concerns on modern GPU architectures [24]. Beyond compute kernels, GPU-enabled asynchronous checkpoint caching and prefetching treat GPU memory as a first-class tier to improve end-to-end throughput for I/O-heavy workflows [35]. Format choices and composition are also critical for irregular workloads, as shown by automatic sparse format composition for SpMM that avoids expensive autotuning while delivering strong performance [40]. Another line of work focuses on reliability, debugging, and cluster-scale GPU management, which together shape how GPU-centric systems are built and operated. GPU-FPX provides low-overhead floating-point exception tracking for NVIDIA GPUs [27]. FPBOXer improves scalable input generation for triggering floating-point exceptions in GPU programs [44], and FloatGuard extends whole-program exception detection to AMD GPUs and highlights cross-vendor portability concerns [36]. At the application and cluster level, SIMCoV-GPU studies multi-node, multi-GPU acceleration for irregular agent-based simulations [25], FASOP automates fast search for near-optimal transformer parallelization on heterogeneous GPU clusters [16], and serverless platforms motivate GPU sharing and scheduling mechanisms such as ESG and FluidFaaS [14, 15]. These works reinforce the importance of aligning work granularity with warps, minimizing synchronization overhead, and designing memory layouts that preserve locality, which are relevant to GPU-resident indexing under dynamic updates.

However, applying these general-purpose HPC optimizations to vector search remains non-trivial due to the irregular memory

access patterns of IVF traversal. SIVF bridges this gap by tailoring these micro-architectural principles, specifically warp-aligned slabs and lock-free synchronization, to construct the first GPU-resident index capable of sustaining high-velocity updates without compromising retrieval integrity.

3 Design and Implementation of SIVF

3.1 Slab-based Memory Management

Standard GPU-based IVF indices rely on contiguous memory arrays to store inverted lists. This monolithic design exhibits severe limitations in streaming scenarios. First, resizing an inverted list requires allocating a larger memory region and copying existing data, introducing synchronization stalls and high latency. Second, frequent insertions and deletions of variable-length lists exacerbate memory fragmentation and reduce allocator efficiency on GPUs.

To address these challenges, we propose a Slab-based Dynamic Memory Allocator (SDMA). Instead of managing variable-length arrays, SDMA pre-allocates a contiguous GPU memory region and manages it as a pool of fixed-size blocks, referred to as *slabs*. Inverted lists are therefore represented as linked chains of slabs. This design transforms dynamic list growth into constant-time block allocation while enabling efficient in-place mutation through lightweight metadata updates.

3.1.1 Slab Layout and Metadata. A *slab* is defined as the fundamental unit of storage. Each slab has a fixed capacity C , and we set $C = 32$ to align with the NVIDIA warp size, thereby enabling coalesced memory access when threads within a warp operate on the same slab. A Slab S_i consists of two regions:

- **Data Payload:** Stores vectors contiguously. For a vector space of dimension D , the payload size is
- **Metadata Header:** Stores control information required for traversal and mutation. The metadata of slab S_i is represented as

$$M_i = \langle n_{next}, b_{valid}, c_{valid} \rangle, \quad (1)$$

where n_{next} denotes the identifier of the next slab in the logical list, b_{valid} is a 32-bit validity bitmap indicating occupied slots, and c_{valid} records the number of valid vectors stored in the slab.

Logical deletion is implemented by atomically clearing the corresponding bit in b_{valid} , yielding constant-time removal without triggering memory compaction or data movement to CPUs.

3.1.2 Conflict-Free Slab Allocation. General-purpose device-side memory allocators incur substantial synchronization and fragmentation overhead when invoked from GPU kernels. SDMA instead manages slabs from a pre-allocated pool. On initialization, a device kernel constructs a free-list array and sets a global stack pointer P_{top} to the pool size. Allocation obtains a fresh slab identifier by atomically decrementing P_{top} :

$$s_{id} = \text{atomicSub}(P_{top}, 1) - 1. \quad (2)$$

In lock-free allocators, reusing memory identifiers may lead to the “ABA” problem, where a shared variable changes from value A

to B and back to A between two observations, causing concurrent threads to falsely assume that the system state remains unchanged. In GPU memory management, this may occur if a slab identifier is reclaimed and reassigned while another thread still holds a stale reference. SDMA avoids this issue by ensuring that slab identifiers are never reused while concurrent kernels may still observe them. Deletion only invalidates logical occupancy via the bitmap and does not immediately recycle slab identifiers back to the pool. This design keeps the meaning of an allocated slab identifier stable during execution and avoids additional versioning mechanisms.

3.1.3 Virtual Addressing via Address Table. To decouple logical vector identifiers from physical storage locations, SDMA introduces a global Address Table that implements a translation function

$$\mathcal{T}(v_{id}) \rightarrow \langle s_{id}, o_{slot} \rangle, \quad (3)$$

where s_{id} denotes the slab identifier and o_{slot} denotes the slot offset within that slab, with $0 \leq o_{slot} < C$.

The address table is initialized with an invalid sentinel value for all entries. Upon insertion, the table is updated to record the physical location of each vector. During deletion, the system queries $\mathcal{T}(v_{id})$ to directly locate the corresponding slab and slot, enabling constant-time invalidation without scanning inverted lists.

3.1.4 System Implementation. We implemented SDMA in CUDA C++ within the Faiss framework using a dual-view architecture. On the host side, GPU memory resources are managed using RAII-compliant containers encapsulated in a SlabManager class, including slab metadata, slab data, address table, and allocator state. These resources are projected into a lightweight SlabManagerDevice structure containing raw pointers and scalar parameters passed by value to device kernels.

Index integration is implemented in GpuIndexSIVF. During initialization, the index selects a safe slab pool size and derives a consistent maximum vector capacity to prevent device memory overflow when slabs are appended. The index maintains an array of per-list head pointers (`list_heads`) initialized to an invalid value. Insertions first perform coarse quantization using the existing Faiss GPU quantizer to obtain list assignments, and then invoke a GPU append kernel that writes vectors into the slab chains. Deletions use the address table to locate and invalidate target slots in place.

Algorithm 1 summarizes the allocator and deletion logic. Insertion resolves the target inverted list, attempts to reserve a free slot in the current head slab, and allocates and links a new slab if needed. The address table is updated after the physical location is finalized. Deletion performs a direct address table lookup followed by an atomic bit clear, leaving slab storage intact.

3.2 Lock-free Parallel Ingestion

Many GPU-based ANN indices are optimized for static or bulk ingestion and commonly rely on sorting, segmented scans, or bulk-copying to build inverted lists. While these approaches can achieve high throughput, they often incur high end-to-end latency for streaming updates. SIVF instead implements a fully parallel insertion kernel in which each CUDA thread ingests one vector and performs lock-free appends to inverted lists. Concurrency is handled using atomic slot reservation in the current head slab and speculative slab expansion under contention. Correctness under

Algorithm 1 Core operations of SDMA

```

1: Global State: slab pool metadata and data, per-list head pointers  $H[\cdot]$ , stack pointer  $P_{top}$ , address table  $\mathcal{T}$ 
2: Input: vector identifier  $v_{id}$ , vector  $x$ , list assignment  $\ell$ 
3: procedure INSERT( $v_{id}, x, \ell$ )
4:    $s \leftarrow H[\ell]$ 
5:   while  $s$  is invalid or no free slot exists in slab  $s$  do
6:      $s_{new} \leftarrow \text{atomicSub}(P_{top}, 1) - 1$ 
7:     Initialize metadata  $s_{new}$  with  $b_{valid} = 0, n_{next} = s$ 
8:     Update  $H[\ell]$  to  $s_{new}$  if unchanged, otherwise retry
9:      $s \leftarrow H[\ell]$ 
10:  end while
11:  Reserve a free slot  $o$  in slab  $s$  via atomic update of  $c_{valid}$ 
12:  Write  $x$  to slab data[ $s$ ][ $o$ ]
13:  Atomically set bit  $o$  in  $b_{valid}$ 
14:   $\mathcal{T}(v_{id}) \leftarrow \langle s, o \rangle$ 
15: end procedure
16: procedure DELETE( $v_{id}$ )
17:    $\langle s, o \rangle \leftarrow \mathcal{T}(v_{id})$ 
18:   if  $\langle s, o \rangle$  is valid then
19:     Atomically clear bit  $o$  in  $b_{valid}$  of slab  $s$ 
20:     Mark  $\mathcal{T}(v_{id})$  as invalid
21:   end if
22: end procedure

```

CUDA’s relaxed memory consistency is ensured by device-wide memory fences and a publish protocol based on a validity bitmap.

At a high level, each SIVF list is represented as a singly linked chain of fixed-capacity slabs. Each slab stores up to $C = 32$ vectors and maintains a counter `valid_count`, a 32-bit `validity_bitmap`, and a link field `next_slab_idx`. Insertion follows a two-stage protocol. First, a thread attempts to reserve a slot in the current head slab. If the head slab is absent or full, the thread allocates a new slab from the global pool and attempts to publish it as the new head using compare-and-swap (CAS). In both cases, the inserted vector becomes visible to concurrent readers only when the corresponding validity bit is set after a memory fence.

3.2.1 Atomic Slot Reservation. Ingestion begins with coarse quantization, assigning each vector to a list ℓ . A thread reads the current head slab index $h = H[\ell]$. If $h \neq -1$, the thread attempts to reserve a writing slot by conditionally incrementing the head slab counter. Let c be the value of `valid_count`. If $c < C$, the thread executes:

$$\text{success} = \text{atomicCAS}(\&\text{slab}[h].\text{valid_count}, c, c + 1) == c. \quad (4)$$

When the above check succeeds, the reserved slot index is $o = c$. This conditional CAS-based increment ensures that exactly one thread claims each slot index and avoids counter overflow under contention. Threads that fail the CAS retry by re-reading the head and counter.

After reserving a slot, the thread writes the vector payload into the slab data region, writes the user identifier into a parallel identifier buffer indexed by $(\text{slab}, \text{slot})$, and updates the address table entry for the user identifier to a compact coordinate encoding $\langle s_{id}, o_{slot} \rangle$. Only after these writes are completed does the thread execute `__threadfence()` and set the corresponding bit in

validity_bitmap via atomicOr. The validity bit is the publication signal for readers, so this ordering ensures that a concurrent search that observes an enabled bit never reads uninitialized payload or a missing address translation.

3.2.2 Speculative Slab Expansion. If the head slab is absent ($h = -1$) or full, the thread expands the list by allocating a new slab from the global pool. Allocation decrements a global stack pointer free_list_top and retrieves a slab index from free_list. If the pool is exhausted, the thread restores the pointer and returns.

After allocation, the thread initializes the new slab metadata by setting valid_count to 1, clearing validity_bitmap to 0, and setting next_slab_idx to the previous head h . A __threadfence() is executed before attempting to publish the new slab as the list head using CAS:

$$\text{success} = \text{atomicCAS}(\&H[\ell], h, s_{\text{new}}) == h. \quad (5)$$

Equation 5 defines the linearization point of list expansion. Only one contending thread succeeds in updating the head pointer, and its newly allocated slab becomes immediately visible to concurrent writers and readers that traverse from the head.

If the head publication CAS fails due to contention, the allocated slab is not returned to the free list. Instead, the thread retries the insertion loop. This leak-on-failure policy avoids additional synchronization on allocator state under contention and prevents hazards that may arise from reclaiming slab identifiers while other threads may still observe stale references. In practice, the overhead is controlled by provisioning sufficient slab pool capacity so that occasional contention-induced leakage does not affect ingestion.

3.2.3 Kernel-Level Memory Ordering and Publication. SIVF enforces safe publication under CUDA’s weak memory consistency using __threadfence() at two critical points. First, before a thread sets a slot’s validity bit, it fences to ensure that payload writes, identifier buffer writes, and address table updates become globally visible. Second, before a thread publishes a newly initialized slab as the list head, it fences to ensure that slab metadata is visible before any concurrent traversal observes the head update. These fences prevent readers from observing a published structural pointer or validity bit that refers to partially initialized state.

3.2.4 Micro-architectural Considerations.

Register-efficient addressing. SIVF represents slab links using 32-bit slab indices within a pre-allocated region rather than 64-bit device pointers. This reduces register pressure and simplifies address computation in the critical path. The address table stores a compact 64-bit coordinate encoding the slab index and slot offset, enabling constant-time deletion without list scans.

Contention behavior and forward progress. Insertion experiences high contention when many vectors map to the same centroid. Most insertions complete through slot reservation when the head slab has available capacity. When the head becomes full, contention shifts to head publication, after which subsequent threads quickly observe the new head and resume slot reservation. The kernel bounds retries with a fixed attempt limit.

Intra-slab coalescing. Vectors within each slab are stored contiguously. When multiple threads append to adjacent slots, payload

Algorithm 2 Lock-free ingestion protocol in SIVF

```

1: Input: vector  $x$ , identifier  $v_{id}$ , list assignment  $\ell$ 
2: Global: head array  $H[\cdot]$ , slab metadata, slab data, free list, stack
   pointer  $P_{top}$ , Address Table  $\mathcal{T}$ 
3:  $attempts \leftarrow 0$ 
4: while  $attempts < \text{MAX\_INSERT\_ATTEMPTS}$  do
5:    $attempts \leftarrow attempts + 1$ 
6:    $h \leftarrow H[\ell]$ 
7:   if  $h \neq -1$  then
8:      $c \leftarrow \text{slab\_meta}[h].\text{valid\_count}$ 
9:     if  $c < C$  then
10:      if  $\text{atomicCAS}(\&\text{slab}[h].\text{valid\_cnt}) == c$  then
11:        Write payload  $x$  to slab data[ $h$ ][ $c$ ]
12:        Write identifier to slab id buffer[ $h$ ][ $c$ ]
13:         $\mathcal{T}(v_{id}) \leftarrow \langle h, c \rangle$ 
14:         $\_\_\text{threadfence}()$ 
15:         $\text{atomicOr}(\&\text{slab\_meta}[h].\text{validity\_bitmap})$ 
16:        return
17:      end if
18:    end if
19:    end if
20:     $t \leftarrow \text{atomicSub}(P_{top}, 1)$ 
21:    if  $t \leq 0$  then
22:       $\text{atomicAdd}(P_{top}, 1)$ 
23:      return
24:    end if
25:     $s_{\text{new}} \leftarrow \text{free\_list}[t - 1]$ 
26:    Set  $\text{slab\_meta}[s_{\text{new}}].\text{valid\_count} \leftarrow 1$ 
27:    Set  $\text{slab\_meta}[s_{\text{new}}].\text{validity\_bitmap} \leftarrow 0$ 
28:    Set  $\text{slab\_meta}[s_{\text{new}}].\text{next} \leftarrow h$ 
29:     $\_\_\text{threadfence}()$ 
30:    if  $\text{atomicCAS}(\&H[\ell], h, s_{\text{new}}) == h$  then
31:      Write payload  $x$  to slab data[ $s_{\text{new}}$ ][0]
32:      Write identifier to slab id buffer[ $s_{\text{new}}$ ][0]
33:       $\mathcal{T}(v_{id}) \leftarrow \langle s_{\text{new}}, 0 \rangle$ 
34:       $\_\_\text{threadfence}()$ 
35:       $\text{atomicOr}(\&\text{slab\_meta}[s_{\text{new}}].\text{validity\_bitmap})$ 
36:      return
37:    end if
38:  end while

```

writes to slab data and identifier buffer writes are naturally coalesced, preserving memory bandwidth under streaming ingestion.

Algorithm 2 summarizes the per-thread ingestion protocol implemented by the insertion kernel. The protocol first attempts slot reservation on the current head slab using CAS on valid_count. If successful, it writes payload, identifier, and address table entry, then publishes the slot by setting the validity bit after a device-wide fence. If the head is absent or full, it allocates and initializes a new slab and attempts to publish it as the new head using CAS on the head pointer, again with a fence before publication. If head publication fails, it retries without reclaiming the allocated slab.

3.3 Coalesced Search on Non-Contiguous Slabs

SIVF replaces contiguous inverted lists with linked slabs, which introduces pointer chasing during traversal. To retain high throughput on GPUs, we design a warp-cooperative search kernel that maps the slab capacity to the warp width. The kernel is optimized for the execution model in which one warp collaboratively processes one query and evaluates one slab per traversal step.

3.3.1 Warp-Cooperative Slab Traversal. SIVF launches the search kernel with one thread block per query and 32 threads per block. This design assigns each query to a single warp. For a query vector $\mathbf{q} \in \mathbb{R}^D$, the warp first stages the query into shared memory to reduce redundant global loads:

$$\mathbf{q} \rightarrow \text{shared_query}.$$

The kernel then probes $nprobe$ coarse lists. For each probed list, the warp traverses the slab chain starting from the list head. At each slab S , thread t_j is responsible for slot j in that slab. The thread consults the slab validity bitmap and only evaluates the distance if the corresponding bit is set:

$$\text{valid}(S, j) \Leftrightarrow ((b_{\text{valid}} \gg j) \& 1) == 1. \quad (6)$$

If valid, the thread loads the vector stored at (S, j) and computes the squared L_2 distance:

$$d(\mathbf{q}, \mathbf{x}_{S,j}) = \sum_{k=1}^D (q_k - x_{S,j,k})^2. \quad (7)$$

The slab pointer is advanced using the metadata `next_slab_idx`. The kernel also includes safety checks that bound the traversal length and break on degenerate self-loops, preventing potential non-terminating traversal under unexpected corruption.

3.3.2 Top- k Maintenance and Warp Aggregation. Maintaining top- k results without global contention is achieved using a two-stage aggregation scheme. Each thread maintains a small top- k list in registers. Upon computing a candidate distance, the thread inserts it into its local top- k list using an in-register insertion routine. This avoids shared data structures and eliminates inter-thread contention during candidate generation.

After all probed lists are traversed, the warp aggregates results. Each thread writes its local top- k list to shared memory. A single thread then performs a deterministic merge across the 32 per-thread lists and outputs the final top- k for the query. This design trades a small amount of serial work for a contention-free aggregation path and avoids additional warp-level primitives.

3.3.3 Search Algorithm. Algorithm 3 describes the proposed search procedure of SIVF based on warp cooperation. We launch one CUDA block per query with 32 threads, so each query is processed by a single warp. At the beginning of the kernel, the warp cooperatively stages the query vector into shared memory to avoid redundant global loads during list traversal. For each query, we probe $nprobe$ coarse lists returned by the quantizer. For a probed list, the warp traverses its slab chain starting from the list head pointer. At each slab, thread t_j corresponds to slot j and consults the slab validity bitmap. If the j -th bit is set, the thread loads the vector payload from the slab data region, computes the squared L_2 distance to the shared query, retrieves the corresponding user identifier from the slab

Algorithm 3 Warp-cooperative search in SIVF

```

1: Input: queries  $Q$ , probed list ids  $coarse\_ids$ , head array  $H[\cdot]$ 
2: Input: slab metadata and data, buffer  $slab\_ids$ ,  $k$ ,  $nprobe$ 
3: Output: top- $k$  distances and labels for each query
4: for each query index  $q$  in parallel do
5:   Launch one warp for query  $q$ 
6:   Copy  $Q[q]$  into shared memory
7:   Initialize per-thread local top- $k$  lists in registers
8:   for  $p = 0$  to  $nprobe - 1$  do
9:      $\ell \leftarrow coarse\_ids[q, p]$ 
10:    if  $\ell$  is invalid then
11:      continue
12:    end if
13:     $s \leftarrow H[\ell]$ 
14:    while  $s$  is valid and traversal bound not exceeded do
15:       $md \leftarrow slab\_meta[s]$ 
16:      if  $md.next = s$  then
17:        break
18:      end if
19:      if  $j < 32$  and  $md.validity\_bitmap[j]$  is set then
20:        Load vector  $\mathbf{x}_{s,j}$  from slab data
21:        Compute  $d(\mathbf{q}, \mathbf{x}_{s,j})$ 
22:         $id \leftarrow slab\_ids[s, j]$ 
23:        Insert  $(d, id)$  into local top- $k$ 
24:      end if
25:       $s \leftarrow md.next$ 
26:    end while
27:  end for
28:  Write local top- $k$  lists to shared memory
29:  One thread merges 32 lists and writes final top- $k$  outputs
30: end for

```

identifier buffer, and updates its local top- k candidates. Traversal proceeds by following the `next_slab_idx` pointer recorded in the slab metadata. To ensure termination under unexpected corruption, we additionally bound the traversal length and break on self-loops.

3.4 Lazy Eviction via Address Translation Table

Traditional vector deletion in IVF indices often requires locating the target element inside an inverted list and physically compacting memory to close the gap, which can incur a cost proportional to the list length and trigger substantial global memory traffic. This overhead fundamentally limits deletion throughput in streaming workloads. SIVF adopts bitmap-based lazy eviction, which decouples logical validity from physical residency. Deletion is reduced to a constant-time metadata update by combining a global Address Translation Table (ATT) with atomic bitwise invalidation of per-slab validity bitmaps.

3.4.1 Address Translation Table (ATT). SIVF maintains a global table $\mathcal{T}$ that maps a user identifier u to its physical coordinate in the slab pool. The table is implemented as a flat array of 64-bit entries. For an identifier u , $\mathcal{T}[u]$ encodes the slab index and slot offset as:

$$\mathcal{T}[u] = (\text{idx}_{slab} \ll 32) \mid \text{idx}_{slot}, \quad (8)$$

where $\text{idx}_{\text{slot}} \in [0, 31]$. The table is initialized with an invalid sentinel value, denoted as `INVALID`, for all entries. This design enables direct random access to the target slot without traversing inverted lists.

3.4.2 Atomic Bitwise Invalidation. Given $\mathcal{T}[u]$, the deletion kernel resolves $(\text{idx}_{\text{slab}}, \text{idx}_{\text{slot}})$ and invalidates the corresponding slot via the slab validity bitmap. Let $\mathcal{B}_i$ denote the 32-bit bitmap of slab S_i . The deletion operation clears the target bit using an atomic read-modify-write:

$$\mathcal{B}_i \leftarrow \text{atomicAnd}(\mathcal{B}_i, \sim (1 \ll \text{idx}_{\text{slot}})). \quad (9)$$

The vector payload is not moved or overwritten. Search kernels consult the bitmap when scanning a slab and ignore slots whose bits are cleared, so deletion immediately removes the vector from the search space without compaction.

3.4.3 Idempotence and Consistency. Deletion requests may include duplicates or may race with other deletions. SIVF treats deletion as an idempotent operation. After the atomic bitmap update, the kernel checks whether the bit was previously set by inspecting the pre-update value returned by `atomicAnd`. Only if the bit transitioned from 1 to 0 does the kernel decrement the slab counter `valid_count` and increment a global deletion counter. Finally, the kernel marks $\mathcal{T}[u] \leftarrow \text{INVALID}$ to prevent future deletion requests from dereferencing a stale coordinate.

This protocol provides a clear correctness contract with concurrent search and insertion. Search considers a slot present only if its validity bit is set. Insertion sets the validity bit only after payload and identifier writes are completed and a device-wide memory fence is executed. Deletion clears the validity bit via an atomic operation, so a deleted slot is excluded from search as soon as the bit is cleared. The ATT invalidation is a secondary safety step that prevents repeated deletions from accessing recycled or stale coordinates. A more formal correctness analysis is provided in Section 3.5.

Deletion performs a constant number of random memory accesses: one read of $\mathcal{T}[u]$, one atomic update of a 32-bit bitmap, and one write of the 64-bit ATT entry to the invalid sentinel. The data payload remains untouched, avoiding bandwidth-heavy compaction. In addition, the implementation optionally updates a 32-bit counter in the slab metadata and a global 32-bit deletion counter for bookkeeping. Overall, the dominant deletion work is independent of the inverted list length, explaining the large speedups observed in our evaluation.

Algorithm 4 presents the bitmap-based lazy eviction procedure implemented by the SIVF deletion kernel. Each GPU thread processes one identifier u in the deletion batch. The thread first performs a constant-time lookup in the ATT to obtain the encoded coordinate $\mathcal{T}[u]$. If the entry equals the invalid sentinel, the identifier is already absent and the thread terminates without side effects. Otherwise, the thread decodes the slab index and slot offset from the 64-bit coordinate, locates the corresponding slab metadata, and clears the target slot by applying an atomic bitwise operation to the slab validity bitmap. The atomic operation returns the previous bitmap value, which allows the kernel to determine whether the bit was previously set. Only when the bit transitions from 1 to 0 does the kernel treat the deletion as successful and perform two bookkeeping updates: it atomically decrements the slab `valid_count`

Algorithm 4 Lazy eviction with ATT in SIVF

```

1: Input: deletion identifiers  $U = \{u_1, \dots, u_m\}$ 
2: Global: Address Table  $\mathcal{T}$ , slab metadata with bitmap  $\mathcal{B}$  and
   counter valid_count
3: for each  $u$  in  $U$  in parallel do
4:    $\text{coord} \leftarrow \mathcal{T}[u]$ 
5:   if  $\text{coord} = \text{INVALID}$  then
6:     continue
7:   end if
8:    $\text{idx}_{\text{slab}} \leftarrow \text{coord} \gg 32$ 
9:    $\text{idx}_{\text{slot}} \leftarrow \text{coord} \& (2^{32} - 1)$ 
10:   $\text{mask} \leftarrow \sim (1 \ll \text{idx}_{\text{slot}})$ 
11:   $\text{old} \leftarrow \text{atomicAnd}(\mathcal{B}_{\text{idx}_{\text{slab}}}, \text{mask})$ 
12:  if  $(\text{old} \gg \text{idx}_{\text{slot}}) \& 1 = 1$  then
13:     $\text{atomicSub}(\&\text{valid\_count}, 1)$ 
14:     $\text{atomicAdd}(\&\text{deleted\_count}, 1)$ 
15:     $\mathcal{T}[u] \leftarrow \text{INVALID}$ 
16:  end if
17: end for

```

and atomically increments a global `deleted_count`. Finally, the kernel writes the invalid sentinel back to $\mathcal{T}[u]$ to prevent future deletion requests from dereferencing a stale coordinate.

3.5 Theoretical Analysis

3.5.1 Correctness. We prove three properties. The first establishes the correctness of parallel ingestion in Algorithm 2. The second shows that search in Algorithm 3 is safe under concurrent ingestion and deletion. The third proves that lazy eviction in Algorithm 4 is linearizable with a clear linearization point.

We use the same state as the pseudocode. For each list id ℓ , $H[\ell]$ stores the head slab id. Each slab s has metadata `slab_meta[s]` including `next`, `valid_count`, and a 32 bit `validity_bitmap`. A slot (s, o) is logically present if and only if bit o in the bitmap is 1. Payloads and ids are stored in `slab_data[s][o]` and `slab_ids[s][o]`. The address table $\mathcal{T}$ maps a vector id u to a coordinate (s, o) encoded in `coord`, or `INVALID`.

Theorem 3.1 (Parallel ingestion is safe and linearizable). *Consider any concurrent execution of Algorithm 2. For every insertion that returns successfully, there exists a unique slot (s, o) such that (i) the insertion writes the payload and id into that slot, then sets bit o in `validity_bitmap` exactly once, (ii) once the bit is set, the slot contains a fully initialized payload and id, and (iii) the insertion can be linearized at the atomic bit set operation `atomicOr` that makes the slot visible.*

PROOF. We first argue the uniqueness of the chosen slot for a successful insertion.

In the existing head case, the code reads

$$c = \text{slab_meta}[h].\text{valid_count}$$

and attempts to reserve index c using an atomic compare and swap. Only one thread can succeed for a given observed c , because the compare and swap updates a single memory location atomically. Therefore, among concurrent contenders for the same slab h , at most one insertion obtains a given index c . In the new slab case, the

thread publishes its new slab as the head using `atomicCAS(&H[l], h, s_new)`; only one thread can win this publication for a fixed old head value h , and the winner uses slot $o = 0$ in its newly allocated slab. Thus, every successful insertion selects exactly one slot (s, o) .

We next argue that the chosen slot is initialized before it becomes visible. In both code paths, after reserving the slot, the writer performs the payload write to `slab_data[s][o]` and the id write to `slab_ids[s][o]`, then writes $\mathcal{T}(v_{id}) \leftarrow \langle s, o \rangle$, then executes `__threadfence()`, and only after the fence performs `atomicOr` to set bit o in the bitmap. The intended CUDA guarantee used here is that the fence makes the prior global writes visible before a later atomic operation performed by the same thread becomes visible to other threads. Therefore, once bit o is observed as set, the payload and id written earlier by that insertion are already visible as well.

Finally, we justify linearizability of a successful insertion with linearization point at the atomic bit set. The membership predicate used throughout the design is exactly the bitmap bit. Before the `atomicOr`, the bit is 0, so the slot is logically absent. After the `atomicOr` takes effect, the bit is 1, so the slot is logically present. Since `atomicOr` is an atomic operation on the bitmap word, it takes effect at a single instant and provides a valid linearization point. At that instant, the slot becomes visible and, by the fence argument above, it is already fully initialized. $\square$

Theorem 3.2 (Search safety under concurrent ingestion and deletion). *In any concurrent execution of Algorithms 2, 3, and 4, every payload and id read performed by Algorithm 3 is fully initialized.*

PROOF. Algorithm 3 reads a slot (s, j) only if it first observes that bit j in `slab_meta[s].validity_bitmap` is set. When the search observes the bit set, the slot must have been published by some successful insertion. By Theorem 3.1, the publication point is the atomic bit set in Algorithm 2, and the payload and id writes occur before the fence, which occurs before the bit set. The same fence guarantee used in Theorem 3.1 implies that once the bit set is visible, the earlier payload and id writes are also visible. Therefore, the search cannot observe a partially initialized payload or id.

Deletion does not write payloads or ids. Algorithm 4 only clears the validity bit and possibly updates counters and $\mathcal{T}$. Hence, even if a search races with a delete, the only effect is whether the search sees the bit as 1 or 0. If it sees 0, it skips the slot and reads nothing. If it sees 1, it reads a payload and id that were fully initialized before publication, by the argument above. In neither case does the search read partially initialized data.

Algorithm 3 also terminates because its traversal is bounded, and it includes an explicit self loop guard (`md.next = s`) before following next. Concurrent insertions can add a new head slab, but they do not require the search to revisit already visited slabs within the traversal bound. $\square$

Theorem 3.3 (Lazy eviction is linearizable and makes deleted vectors invisible). *Consider any concurrent execution of Algorithms 3 and 4. For any delete request on id u such that $\mathcal{T}[u] \neq \text{INVALID}$ and decodes to (idx_{slab}, idx_{slot}) , define*

$$mask = \sim (1 \ll idx_{slot}).$$

Then the atomic operation $old = \text{atomicAnd}(\&\mathcal{B}_{idx_{slab}}, mask)$ in Algorithm 4 is a linearization point for logical deletion. After this

atomic operation takes effect, the slot becomes logically absent from the search space until a later insertion sets the same bit again. Repeated deletes of the same id are safe.

PROOF. If $\mathcal{T}[u] = \text{INVALID}$, Algorithm 4 performs no state change and the delete can be linearized at the `INVALID` check.

Otherwise, the delete decodes `coord` into (idx_{slab}, idx_{slot}) , computes $mask = \sim (1 \ll idx_{slot})$, and executes

$$old = \text{atomicAnd}(\&\mathcal{B}_{idx_{slab}}, mask)$$

to clear the corresponding bit. The intended guarantee is that this atomic read modify write takes effect at a single instant on the bitmap word. We choose that instant as the linearization point.

Algorithm 3 uses the validity bit as the membership predicate and checks it before dereference. Thus, after the clear takes effect, any search that reads the bitmap and sees the bit as 0 skips the slot and does not include it as a candidate. No payload reclamation is required because logical membership is determined solely by the validity bit.

If the old bitmap value shows the bit transitioned from 1 to 0, the delete performs bookkeeping and sets $\mathcal{T}[u] \leftarrow \text{INVALID}$. If the bit was already 0, clearing it again does not change the logical state. Either way, repeated deletes are safe. $\square$

3.5.2 Time Complexity. We analyze the complexity relative to the number of existing vectors N and the dimensionality D . For ingestion, the standard dynamic array approach requires $O(N)$ time in the worst case to copy data during capacity resizing. In contrast, the slab-based allocation in SIVF operates in $O(1)$ time, as it involves only pointer manipulation and atomic counter updates, independent of the current list size. For deletion, traditional IVF implementations without a reverse index require scanning the target inverted list, yielding a complexity of $O(N/n_{list})$. Even with a reverse index, physical removal requires $O(N/n_{list})$ data movement to maintain array contiguity. SIVF achieves $O(1)$ deletion complexity by utilizing the Address Translation Table to locate the target slot in constant time and performing a logical invalidation via a single atomic instruction. For search, the complexity remains $O(n_{probe} \cdot (N/n_{list}) \cdot D)$, which is asymptotically equivalent to standard IVF. While the linked-slab layout introduces pointer-chasing overhead, it does not alter the algorithmic complexity class.

3.5.3 Space Complexity. The space complexity of SIVF consists of the vector storage, the slab metadata, and the address translation structures. Let S be the slab capacity (fixed at 32). The metadata overhead per vector is constant, scaling as $O(1/S)$. The Address Translation Table introduces a linear space overhead of $O(N_{max})$, where each entry requires 8 bytes. Comparing this to standard dynamic arrays, which often reserve up to $2 \times N$ capacity to amortize resizing costs, SIVF maintains a tighter memory footprint. The external fragmentation is bounded by the size of the free pool, and internal fragmentation is limited to the unused slots in the tail slab of each list. Consequently, the total space complexity is $O(N \cdot D)$, maintaining linear scalability with dataset size.

4 Evaluation

[Dongfang: Add CAGRA results]

[Dongfang: Add multi-GPU results]

[Dongfang: Remove GIST1M results]

We implement SIVF by extending the GPU components of the widely-adopted Faiss library [6], integrating our proposed data structures and algorithms into its core indexing architecture. Our evaluation benchmarks SIVF directly against the native Faiss GPU implementations, specifically targeting ingestion throughput and deletion latency under high-concurrency streaming workloads. We have open-sourced our complete implementation and evaluation scripts on GitHub.

Implementing SIVF required substantial modifications to the Faiss GPU backend and associated experimental infrastructure. In total, the implementation involves approximately 9,000 lines of code changes across more than 45 source files. Core system changes are concentrated in the GPU index implementation, incorporating new GPU-native insertion, deletion, and search operators (e.g., `GpuIndexSIVF`, `SIVFAppend`, `SIVFSearch`, `SIVFDelete`), alongside a custom slab-based memory manager (e.g., `SlabManager`) and lock-free metadata structures (e.g., `SIVFStructs`). Furthermore, we developed a comprehensive benchmarking suite that extends beyond standard metric testing to include end-to-end streaming simulations, memory scalability analysis, and automated visualization scripts, ensuring a rigorous and reproducible evaluation.

4.1 Experimental Setup

To ensure a comprehensive evaluation, we adopt a two-tiered baseline strategy. First, our primary baseline is the standard Faiss GPU IVF, which serves as a direct architectural counterpart; this allows us to isolate the performance gains specifically attributable to our proposed memory management innovations, excluding confounding factors from differing algorithmic paradigms. Second, to position SIVF within the broader indexing landscape, we in Section 4.7 compare against representative non-IVF indices, including GPU Flat (brute-force), HNSW [34] (dynamic graph), NSG [8] (static graph), and LSH [4] (hashing).

All experiments were conducted on a bare-metal node from the Chameleon Cloud testbed [23] (CHI@UC site). The platform is equipped with dual-socket Intel Xeon Gold 6126 CPUs (Skylake microarchitecture), providing a total of 24 physical cores (48 threads with Hyper-Threading) clocked at 2.60 GHz. The host memory consists of 192 GiB of RAM. For acceleration, the node features an NVIDIA Quadro RTX 6000 GPU with 24 GB of GDDR6 VRAM. The system runs on a Ubuntu 24.04 environment with the CUDA toolkit installed: Driver ver. 560.35.05 and CUDA ver. 12.2.

Each reported data point represents the average of at least three independent executions. Error bars are omitted in figures where the standard deviation is negligible to maintain visual clarity. Unless explicitly stated (as in Section 4.4), our experiments utilize synthetic datasets consisting of random 128-dimensional vectors generated from a uniform distribution. We employ this configuration as a neutral baseline to isolate system-level overheads (e.g., memory allocation, PCIe transfer) from data distribution effects. Note that because uniform data lacks the clustering structure inherent in real-world applications, the performance advantages of SIVF in microbenchmarks are primarily architectural and appear more conservative than real-world datasets.

4.2 Microbenchmarks

4.2.1 Ingestion. We evaluate ingestion throughput against the standard Faiss GPU IVFFlat baseline, varying database size (N_B : 1M–4M) and cluster count (n_{list} : 1024–16384). As shown in Figure 2, SIVF achieves consistent performance advantages with up to 2.65× speedup.

Scalability (Fig. 2a). SIVF maintains a stable throughput of ≈ 4.2 M vec/s as N_B increases, significantly outperforming the baseline (≈ 1.7 M vec/s). This stability validates our lock-free SlabManager, which ensures $O(1)$ insertion cost independent of index occupancy, thereby avoiding the contiguous memory resizing overheads inherent to the baseline.

Granularity Impact (Fig. 2b). While the baseline throughput degrades monotonically as n_{list} increases (dropping from 2.2M to 1.7M vec/s), SIVF exhibits a performance sweet spot at $n_{list} = 2048$, peaking at 5.3M vec/s. At this optimal granularity, SIVF achieves a 2.65× speedup over the baseline. Even at high $n_{list} = 4096$, where scattered memory writes typically hinder performance, SIVF maintains robust throughput at 4.2M vec/s, sustaining a 2.47× advantage over the baseline’s 1.7M vec/s.

Overall Speedup (Fig. 2c). SIVF outperforms the baseline across all configurations, with speedups typically ranging from 2.4× to 2.9×. Even under maximum contention (4M vectors, $n_{list} = 1024$), SIVF retains a 1.79× advantage, confirming the efficiency of the non-blocking, slab-based pipeline for streaming scenarios.

4.2.2 Search. We evaluate query performance against the Vanilla Faiss GPU baseline. While SIVF trades memory contiguity for mutability, Figure 3 confirms competitive efficiency, reaching up to 91% of baseline performance in optimized configurations.

Scalability (Fig. 3a). SIVF throughput scales predictably from ≈ 383 K to 118K QPS as database size grows (100K–500K, $n_{list} = 4096$). The trend closely mirrors the baseline, validating the stability of our slab traversal logic under increasing load.

Granularity Impact (Fig. 3b). Increasing n_{list} from 1024 to 4096 boosts SIVF throughput by 69% (70K $\rightarrow$ 118K QPS), effectively balancing the reduced search space per probe against memory access overhead. However, at finer granularity ($n_{list} = 16384$), performance slightly regresses to 100K QPS. This indicates that while finer clusters reduce distance computations within lists, the overhead of managing excessive lists and reduced memory locality eventually begins to diminish returns.

Query Efficiency (Fig. 3c). The efficiency ratio, which is defined as $(QPS_{SIVF}/QPS_{Vanilla})$, highlights the effectiveness of warp-level optimizations. In sparse scenarios ($n_{list} = 16384$, 100K vectors), SIVF achieves 0.91× efficiency, nearly matching contiguous arrays. As lists lengthen (500K vectors), the ratio stabilizes between 0.40× and 0.52×, reflecting the necessary physical trade-off of pointer chasing for dynamic memory support.

4.2.3 Deletion. We evaluate the efficiency of the vector deletion mechanism in SIVF by comparing it against the standard Faiss GPU IVF baseline. The experiment measures latency and throughput

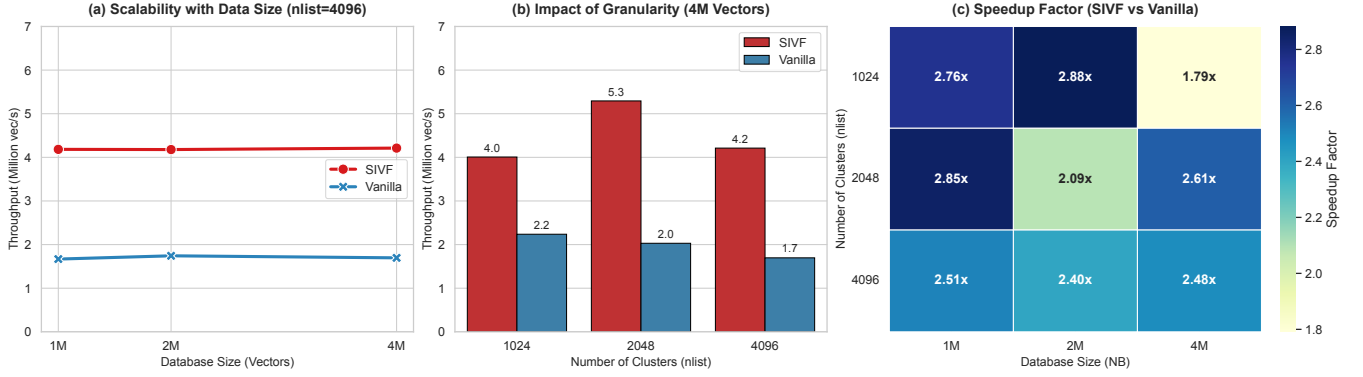


Figure 2: Microbenchmark performance evaluation of vector ingestion.

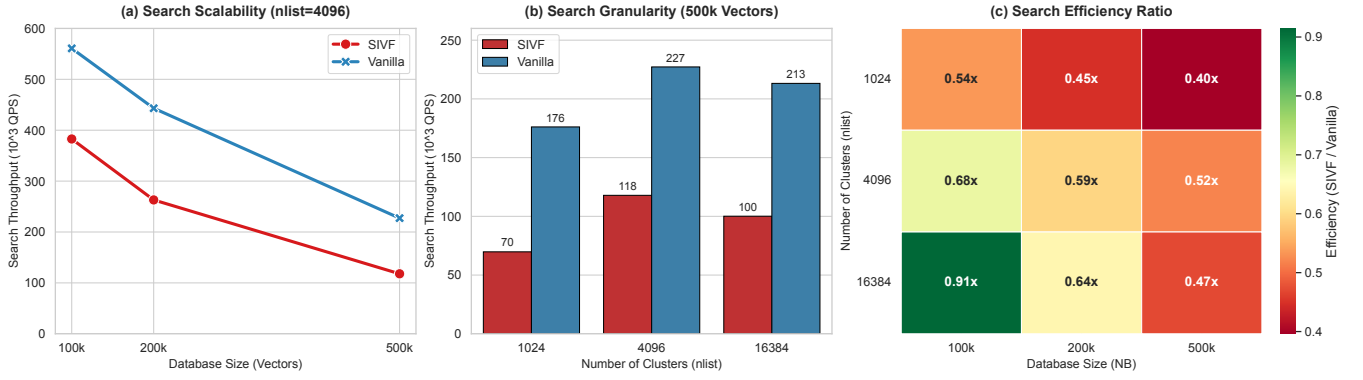


Figure 3: Microbenchmark performance evaluation of vector search.

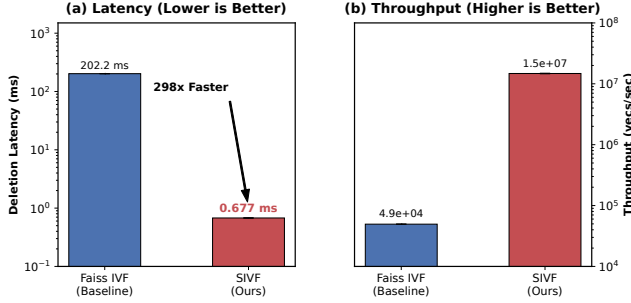


Figure 4: Microbenchmark performance of vector deletion.

when removing a batch of 10,000 vectors from a populated index of one million 128-dimensional vectors.

Figure 4 illustrates the performance comparison between the two methods. The results indicate that the baseline Faiss implementation incurs a substantial deletion latency of approximately 202.2 ms. In stark contrast, SIVF completes the same batch deletion operation in an average of 0.68 ms. This represents a massive speedup of 298.5 $\times$. Correspondingly, deletion throughput surges from 4.9×10^4 vec/s in the baseline to nearly 1.5×10^7 vec/s in SIVF.

4.3 Parameter Sensitivity and Ablation Study

We evaluate system robustness by sweeping three parameters: (i) vector capacity factor (*maxvec_factor*, or *mv*), (ii) slab pool redundancy (*slab_factor*, or *sl*), and (iii) deletion batch size (*b*). These control logical headroom, pre-allocated memory for bursts, and the trade-off between amortization and freshness. Figures 5 and 6 summarize the results.

Impact of Pre-allocation Strategy (Fig. 5a). Insertion throughput correlates positively with memory provisioning. Increasing *maxvec_factor* from 1.1 to 1.5 yields a clear performance uplift (e.g., 2.71 $\rightarrow$ 2.95 M vec/s), as larger initial capacities reduce the frequency of dynamic resizing. Similarly, a larger batch size (*b* = 8192) consistently outperforms smaller batches, culminating in a peak throughput of 3.23M vec/s. This confirms that SIVF benefits from amortizing kernel launch overheads via batching and minimizing allocation contention via generous pre-allocation.

Deletion Stability and Resource Constraints (Fig. 5b). Deletion performance reveals a tradeoff between resource provisioning and batching efficiency. Under tight memory constraints (*mv* : 1.1, *sl* : 1.0), throughput is highly sensitive to batch size, dropping to 1.08M vec/s with small batches due to allocation contention. However, relaxing memory constraints by increasing *slab_factor* to 1.3 or

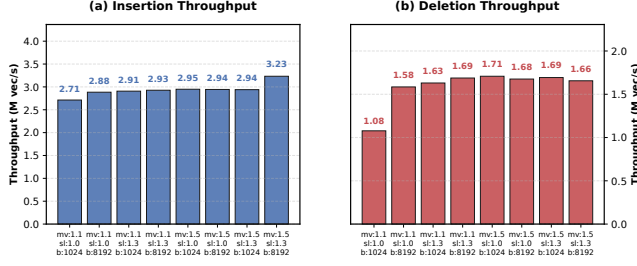


Figure 5: Throughput sensitivity analysis.

$maxvec_factor$ to 1.5 eliminates this bottleneck, stabilizing throughput around 1.7M vec/s regardless of batch size. This demonstrates that SIVF’s slab allocator effectively decouples deletion latency from fragmentation when sufficiently provisioned.

Latency Stability (Fig. 6). Figure 6 analyzes the end-to-end latency for index mutations. Insertion latency (Fig. 6a) remains consistent between 3.09 ms and 3.69 ms. Performance correlates with memory provisioning: increasing $maxvec_factor$ to 1.5 minimizes dynamic resizing overhead, achieving the lowest insertion latency of 3.09 ms. Deletion operations (Fig. 6b) are significantly faster, maintaining sub-millisecond latency (0.58–0.93 ms) across all settings. This 3×–5× speed advantage over insertion stems from SIVF’s lock-free bitmap mechanism, which avoids the data movement required for insertion. Notably, deletion latency benefits from batch amortization, dropping from 0.93 ms to 0.63 ms as batch size increases, confirming that kernel launch overhead dominates the cost of these lightweight bitwise operations.

4.4 Real-world Datasets

We benchmark SIVF using four popular real-world datasets representing diverse modalities: Deep1B [50] (96 dimensions, deep features), SIFT1M [22] (128 dimensions, vision), T2I-1B [42] (200 dimensions, multimodal/RAG), and GIST1M [22] (960 dimensions, high-dim features). While the subset size (1M vectors) fits within GPU memory, these experiments validate performance within the *active window* of streaming applications. Section 4.5 will report their stability under high-churn workloads.

High-Throughput Ingestion. As shown in Figure 7, SIVF dominates ingestion scalability across all dimensions. On Deep1B, SIVF achieves a peak throughput of 4.38M vec/s (120× speedup over the baseline’s 36.4K vec/s). This advantage persists on SIFT1M (3.78M vec/s, 105×) and T2I-1B (2.91M vec/s, 84×). Even on high-dimensional GIST1M, SIVF maintains 852.7K vec/s (36× speedup), confirming that our pre-allocated slab architecture effectively saturates GPU bandwidth by eliminating dynamic resizing overhead.

Low-Latency Deletion. Figure 8 highlights the critical performance divergence. Baseline latency scales poorly due to CPU-GPU roundtrips, varying from 1.2s (Deep1B) to 11.8s (GIST1M). In contrast, SIVF’s in-place bitmap updates decouple latency from dimension, maintaining sub-millisecond performance (< 0.9 ms) across all datasets. Notably on GIST1M, SIVF reduces latency from 11,843



Figure 6: Latency sensitivity analysis.

ms to 0.89 ms, a 13,307× improvement, validating its feasibility for real-time streams.

Search Performance and Trade-offs. Figure 9 compares query throughput (QPS). SIVF exhibits a lower-dimension advantage: on Deep1B, it outperforms the baseline by 2.07× (59.8K vs. 28.9K QPS) while maintaining comparable recall (91.9% vs 92.0%, not shown in the figure). On SIFT1M, it retains a 1.5× lead. Performance parity is observed on T2I-1B (17.8K vs 18.6K QPS). A trade-off appears on high-dimensional GIST1M, where SIVF achieves 37% of baseline QPS. This results from reduced memory coalescing efficiency when traversing non-contiguous slabs with large vectors, which is a necessary compromise to enable instant, lock-free mutability.

4.5 End-to-End Streaming Performance

We evaluate real-time applicability using a *Sliding Window* benchmark that mimics production streams. The system maintains a fixed active window (W) by ingesting a new batch (B) and evicting the oldest batch (B) at each step. We test on SIFT1M ($W = 200K, B = 10K$) and GIST1M ($W = 100K, B = 5K$).

As shown in Figure 10, the standard Faiss baseline suffers from high latency due to the CPU-GPU roundtrip required for index reconstruction, causing system freezes of 355 ms (SIFT1M) and > 1.1 s (GIST1M) per update. In contrast, SIVF executes updates strictly in VRAM via slab-based management, reducing per-step latency to ≈ 2.2 ms (SIFT1M) and 4.2 ms (GIST1M). This yields speedups of 163× and 262×, respectively. Notably, the performance gap widens with dimensionality (GIST1M), where the baseline is bottlenecked by PCIe saturation. SIVF eliminates this bottleneck, maintaining single-digit millisecond latency and transforming the GPU IVF index into a real-time streaming engine.

4.6 Memory Efficiency and Scalability

A common concern with dynamic graph-based or list-based indices is the potential for memory bloating due to structural overhead, such as pointers and metadata, as well as fragmentation. To quantify the space complexity of SIVF, we conducted a memory footprint analysis comparing the allocated VRAM usage of SIVF against a theoretically compact array baseline across varying dataset sizes ranging from 100K to 1M vectors.

Figure 11 illustrates the memory growth trends for both SIFT1M and GIST1M. The results demonstrate that SIVF exhibits deterministic linear scalability with negligible structural overhead. For SIFT1M ($d = 128$), the overhead stabilizes at 0.77%, while for the

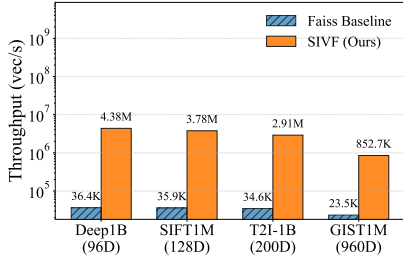


Figure 7: Ingestion throughput.

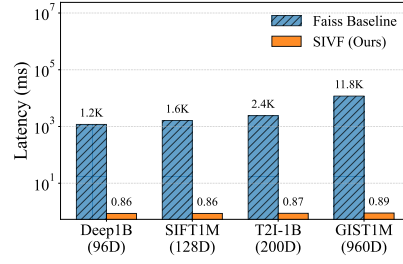


Figure 8: Deletion latency.

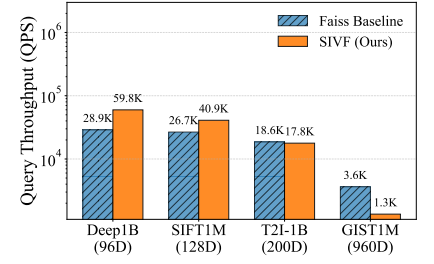


Figure 9: Search performance (QPS).

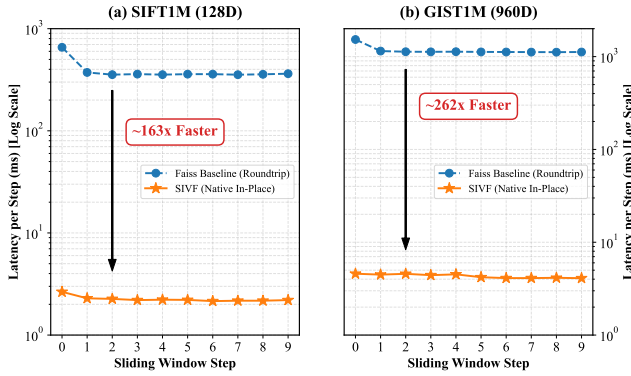


Figure 10: End-to-End Streaming Performance.

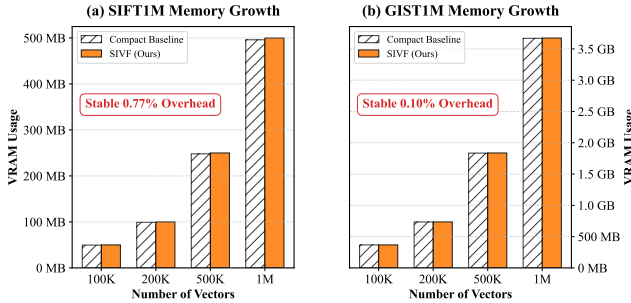


Figure 11: Memory Efficiency and Scalability.

high-dimensional GIST1M ($d = 960$), it is merely 0.10%. This efficiency stems from our coarse-grained slab design, where the meta-data cost (128-byte header) is amortized over a batch of 32 vectors. Consequently, SIVF achieves orders-of-magnitude performance gains in dynamic operations without imposing significant storage penalties compared to static, compact indices.

Furthermore, we verified the efficacy of our memory reclamation strategy. In a stress test (not shown in the figure) involving the deletion of 50% of the dataset followed by immediate re-insertion, SIVF maintained a constant memory footprint without triggering OS-level deallocation. The deletion operation completed in sub-millisecond time per batch (e.g., 4.04 ms for 100K GIST vectors), confirming that memory slots are logically reclaimed via bitmap

Table 1: Comparison of ingestion throughput (K vec/s) and deletion latency (ms) across different datasets.

Method	SIFT1M		T2I-1B		GIST1M	
	Add	Del	Add	Del	Add	Del
GPU Flat [6]	9,169	836	6,020	1,160	1,230	1,143
HNSW [34]	25.5	N/A	25.7	N/A	0.6	N/A
NSG [8]	3.7	N/A	3.2	N/A	0.7	N/A
LSH [4]	787	14.6	428	16.3	36.6	9.2
SIVF (this work)	3,780	0.86	2,910	0.87	853	0.89

toggling and immediately available for reuse. This “zero-cost” reclamation ensures that SIVF can sustain long-running streaming workloads without suffering from memory leaks or fragmentation-induced bloat.

4.7 SIVF vs. Non-IVF Streaming

To position SIVF within the broader indexing landscape, we benchmarked it against four representative non-IVF architectures: (1) Faiss GPU Flat (brute-force baseline, i.e., no index), (2) HNSW [34] (dynamic graphs), (3) LSH [4] (legacy hash-based approaches), and (4) NSG [8] (static graphs).

Table 1 reports the results of both vector ingestion and deletion in a streaming context on three datasets. graph indices (HNSW, NSG) suffer catastrophic performance degradation on high-dimensional data, dropping to merely 600 vec/s on GIST1M due to cache thrashing, and fail to support native deletion (N/A). While GPU Flat achieve the highest ingestion throughput because they perform brute-force search without indexing overhead, they incur prohibitive deletion latencies (> 1 s) due to CPU-GPU roundtrips. SIVF emerges as the only architecture enabling holistic stream processing: it sustains GPU-class ingestion rates (e.g., 853K vec/s on GIST1M, orders of magnitudes faster than graphs) while providing sub-millisecond native deletion, effectively bridging the gap between static high-throughput search and dynamic mutability.

5 Conclusion

This work addresses a structural limitation inherent in the de facto GPU-accelerated Inverted File (IVF) indices, which effectively precludes their use in streaming vector analytics due to rigid and static memory layouts. A new GPU-native indexing method, namely SIVF, is proposed, which relocates memory management responsibilities

entirely onto the device. SIVF has been implemented and integrated into the industry-standard Faiss library; extensive evaluation spanning 5 baseline architectures and 5 datasets demonstrates that SIVF effectively bridge the latency/throughput gap between static high-throughput search and dynamic mutability with negligible memory overhead in streaming vector analytics.

Future Work. The current implementation operates on a single GPU, bounding the maximum active window size to the available VRAM. While our sliding window experiments demonstrate stability under infinite streams, scaling the *simultaneous* index capacity to larger datasets will require extending SIVF with *multi-GPU sharding* via NVLink and unified memory prefetching techniques.

References

- [1] CHATZAKIS, M., PAPA-KONSTANTINOY, Y., AND PALPANAS, T. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data* 3, 4 (Sept. 2025).
- [2] CHU, G., HE, Y., DONG, L., DING, Z., CHEN, D., BAI, H., WANG, X., AND HU, C. Efficient algorithm design of optimizing spmv on gpu. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 115–128.
- [3] CHUNG, J. W., LIN, H., AND ZHAO, W. Locality-sensitive indexing for graph-based approximate nearest neighbor search. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2418–2428.
- [4] DATAR, M., IMMORLICA, N., INDYK, P., AND MIRROKNI, V. S. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry* (New York, NY, USA, 2004), SCG '04, Association for Computing Machinery, p. 253–262.
- [5] DENG, L., CHEN, P., ZENG, X., WANG, T., ZHAO, Y., AND ZHENG, K. Efficient data-aware distance comparison operations for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.* 18, 3 (Nov. 2024), 812–821.
- [6] DOUZE, M., GUZHVA, A., DENG, C., JOHNSON, J., SZILVASY, G., MAZARÉ, P.-E., LOMELI, M., HOSSEINI, L., AND JÉGOU, H. The faiss library.
- [7] ENGELS, J., LANDRUM, B., YU, S., DHULIPALA, L., AND SHUN, J. Approximate nearest neighbor search with window filters. In *Forty-first International Conference on Machine Learning* (2024).
- [8] FU, C., XIANG, C., WANG, C., AND CAI, D. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.* 12, 5 (Jan. 2019), 461–474.
- [9] GAO, J., GOU, Y., XU, Y., YANG, Y., LONG, C., AND WONG, R. C.-W. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search. *Proc. ACM Manag. Data* 3, 3 (June 2025).
- [10] GAO, J., AND LONG, C. Rabbitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proc. ACM Manag. Data* 2, 3 (May 2024).
- [11] GONG, Z., ZENG, Y., AND CHEN, L. Accelerating approximate nearest neighbor search in hierarchical graphs: Efficient level navigation with shortcuts. *Proc. VLDB Endow.* 18, 10 (June 2025), 3518–3530.
- [12] GOU, Y., GAO, J., XU, Y., AND LONG, C. Symphonyqg: Towards symphonious integration of quantization and graph for approximate nearest neighbor search. *Proc. ACM Manag. Data* 3, 1 (Feb. 2025).
- [13] HUA, Z., MO, Q., YAO, Z., CUI, L., LIU, X., WANG, G., WEI, Z., LIU, X., TANG, T., LIU, S., AND QU, L. Dynamically detect and fix hardness for efficient approximate nearest neighbor search. *Proc. ACM Manag. Data* 3, 6 (Dec. 2025).
- [14] HUI, X., XU, Y., GUO, Z., AND SHEN, X. Esg: Pipeline-conscious efficient scheduling of dnn workflows on serverless platforms with shareable gpus. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 42–55.
- [15] HUI, X., XU, Y., AND SHEN, X. Fluidfaas: A dynamic pipelined solution for serverless computing with strong isolation-based gpu sharing. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.
- [16] HWANG, S., LEE, E., OH, H., AND YI, Y. Fasop: Fast yet accurate automated search for optimal parallelization of transformers on heterogeneous gpu clusters. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 253–266.
- [17] INDYK, P., AND XU, H. Worst-case performance of popular approximate nearest neighbor search implementations: Guarantees and limitations. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.
- [18] JÄÄSAARI, E., HYVÖNEN, V., AND ROOS, T. Lorann: Low-rank matrix factorization for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.
- [19] JIANG, M., YANG, Z., ZHANG, F., HOU, G., SHI, J., ZHOU, W., LI, F., AND WANG, S. Digra: A dynamic graph indexing for approximate nearest neighbor search with range filter. *Proc. ACM Manag. Data* 3, 3 (June 2025).
- [20] JOHNSON, J., DOUZE, M., AND JÉGOU, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.
- [21] JUNG, S., PARK, Y., LEE, H., OH, Y. H., AND LEE, J. W. Angular distance-guided neighbor selection for graph-based approximate nearest neighbor search. In *Proceedings of the ACM on Web Conference 2025* (New York, NY, USA, 2025), WWW '25, Association for Computing Machinery, p. 4014–4023.
- [22] JÉGOU, H., DOUZE, M., AND SCHMID, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.
- [23] KEAHEY, K., ANDERSON, J., ZHEN, Z., RITEAU, P., RUTH, P., STANZIONE, D., CEVIK, M., COLLIERAN, J., GUNAWI, H. S., HAMMOCK, C., MAMBRETTI, J., BARNES, A., HALBACH, F., ROCHA, A., AND STUBBS, J. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.
- [24] LAUT, S., BORRELL, R., AND CASAS, M. Extending sparse patterns to improve inverse preconditioning on gpu architectures. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 200–213.
- [25] LEYBA, K., HOFMEYER, S., FORREST, S., CANNON, J., AND MOSES, M. Simcov-gpu: Accelerating an agent-based model for exascale. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 322–333.
- [26] LI, M., YAN, X., LU, B., ZHANG, Y., CHENG, J., AND MA, C. Attribute filtering in approximate nearest neighbor search: An in-depth experimental study. *Proc. ACM Manag. Data* 3, 6 (Dec. 2025).
- [27] LI, X., LAGUNA, I., FANG, B., SWIRYDOWICZ, K., LI, A., AND GOPALAKRISHNAN, G. Design and evaluation of gpu-fpx: A low-overhead tool for floating-point exception detection in nvidia gpus. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 59–71.
- [28] LI, Z., KE, X., ZHU, Y., YU, B., ZHENG, B., AND GAO, Y. Scalable graph indexing using gpus for approximate nearest neighbor search. *Proc. ACM Manag. Data* 3, 6 (Dec. 2025).
- [29] LI, Z., KE, X., ZHU, Y., YU, B., ZHENG, B., AND GAO, Y. Scalable graph indexing using gpus for approximate nearest neighbor search. In *Proceedings of the 2026 International Conference on Management of Data (SIGMOD)* (2026).
- [30] LIANG, A., ZHANG, P., YAO, B., CHEN, Z., SONG, Y., AND CHENG, G. Unify: Unified index for range filtered approximate nearest neighbors search. *Proc. VLDB Endow.* 18, 4 (Dec. 2024), 1118–1130.
- [31] LIU, M., ZHONG, S., YANG, Q., HAN, Y., LIU, X., AND MA, Y. Webanns: Fast and efficient approximate nearest neighbor search in web browsers. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2483–2492.
- [32] LU, D., FENG, S., ZHOU, J., SOLLEZA, F., SCHWARZKOPF, M., AND ÇETINTEMEL, U. Vectraflow: Integrating vectors into stream processing. In *Proceedings of the 2025 Conference on Innovative Data Systems Research (CIDR)* (Jan. 2025).
- [33] LU, K., XIAO, C., AND ISHIKAWA, Y. Probabilistic routing for graph-based approximate nearest neighbor search. In *Forty-first International Conference on Machine Learning* (2024).
- [34] MALKOV, Y. A., AND YASHUNIN, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (Apr. 2020), 824–836.
- [35] MAURYA, A., RAFIQUE, M. M., TONELLO, T., ALSALEM, H. J., CAPPELLO, F., AND NICOLAE, B. Gpu-enabled asynchronous multi-level checkpoint caching and prefetching. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 73–85.

- [36] MIAO, D., LAGUNA, I., AND RUBIO-GONZÁLEZ, C. Floatguard: Efficient whole-program detection of floating-point exceptions in amd gpus. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.
- [37] MOHONEY, J., SARDA, D., TANG, M., CHOWDHURY, S. R., PACACI, A., ILYAS, I. F., REKATSINAS, T., AND VENKATARAMAN, S. Quake: adaptive indexing for vector search. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation* (USA, 2025), OSDI '25, USENIX Association.
- [38] OOTOMO, H., NARUSE, A., NOLET, C., WANG, R., FEHER, T., AND WANG, Y. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)* (Los Alamitos, CA, USA, May 2024), IEEE Computer Society, pp. 4236–4247.
- [39] PENG, Z., QIAO, M., ZHOU, W., LI, F., AND DENG, D. Dynamic range-filtering approximate nearest neighbor search. *Proc. VLDB Endow.* 18, 10 (June 2025), 3256–3268.
- [40] PENG, Z., THOMADAKIS, P., PIENAAR, J., AND KESTOR, G. Liteform: Lightweight and automatic format composition for sparse matrix-matrix multiplication on gpus. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.
- [41] QIU, R., AND TANG, J. Efficient approximate nearest neighbor search via hemisphere centroids graph. *Proc. ACM Manag. Data* 3, 6 (Dec. 2025).
- [42] SIMHADRI, H. V., AUMÜLLER, M., DOUZE, M., BARANCHUK, D., INGBER, A., LIBERTY, E., WILLIAMS, G., LANDRUM, B., MANOHAR, M. D., KARJIKAR, M., DHULIPALA, L., CHEN, M., CHEN, Y., MA, R., ZHANG, K., CAI, Y., SHI, J., ZHENG, W., CHEN, Y., YIN, J., AND HUANG, B. Results of the big ANN: NeurIPS'23 competition. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track* (2025).
- [43] SUN, P., SIMCHA, D., DOPSON, D., GUO, R., AND KUMAR, S. SOAR: improved indexing for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.
- [44] TRAN, A., LAGUNA, I., AND GOPALAKRISHNAN, G. Fpboxer: Efficient input-generation for targeting floating-point exceptions in gpu programs. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 83–93.
- [45] VORUGANTI, S., AND ÖZSU, M. T. Mirage-anns: Mixed approach graph-based indexing for approximate nearest neighbor search. *Proc. ACM Manag. Data* 3, 3 (June 2025).
- [46] WANG, M., LV, L., XU, X., WANG, Y., YUE, Q., AND NI, J. An efficient and robust framework for approximate nearest neighbor search with attribute constraint. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.
- [47] WANG, Z., ZHANG, J., AND HU, W. Wow: A window-to-window incremental index for range-filtering approximate nearest neighbor search. *Proc. ACM Manag. Data* 3, 6 (Dec. 2025).
- [48] WEI, J., LEE, X., LIAO, Z., PALPANAS, T., AND PENG, B. Subspace collision: An efficient and accurate framework for high-dimensional approximate nearest neighbor search. *Proc. ACM Manag. Data* 3, 1 (Feb. 2025).
- [49] XU, Q., ZHANG, F., LI, C., CAO, L., CHEN, Z., ZHAI, J., AND DU, X. Harmony: A scalable distributed vector database for high-throughput approximate nearest neighbor search. *Proc. ACM Manag. Data* 3, 4 (Sept. 2025).
- [50] YANDEX, A. B., AND LEMPITSKY, V. Efficient indexing of billion-scale datasets of deep descriptors. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016), pp. 2055–2063.
- [51] YANG, M., CAI, Y., AND ZHENG, W. CSPG: crossing sparse proximity graphs for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.
- [52] ZHANG, B., TIAN, J., DI, S., YU, X., FENG, Y., LIANG, X., TAO, D., AND CAPPELLO, F. Fz-gpu: A fast and high-ratio lossy compressor for scientific computing applications on gpus. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 129–142.
- [53] ZHANG, F., JIANG, M., HOU, G., SHI, J., FAN, H., ZHOU, W., LI, F., AND WANG, S. Efficient dynamic indexing for range filtered approximate nearest neighbor search. *Proc. ACM Manag. Data* 3, 3 (June 2025).
- [54] ZHANG, Z., LIU, F., HUANG, G., LIU, X., AND JIN, X. Fast vector query processing for large datasets beyond GPU memory with reordered pipelining. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)* (Santa Clara, CA, Apr. 2024), USENIX Association, pp. 23–40.
- [55] ZHONG, X., LI, H., JIN, J., YANG, M., CHU, D., WANG, X., SHEN, Z., JIA, W., GU, G., XIE, Y., LIN, X., SHEN, H. T., SONG, J., AND CHENG, P. Vsag: An optimized search framework for graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 18, 12 (Aug. 2025), 5017–5030.